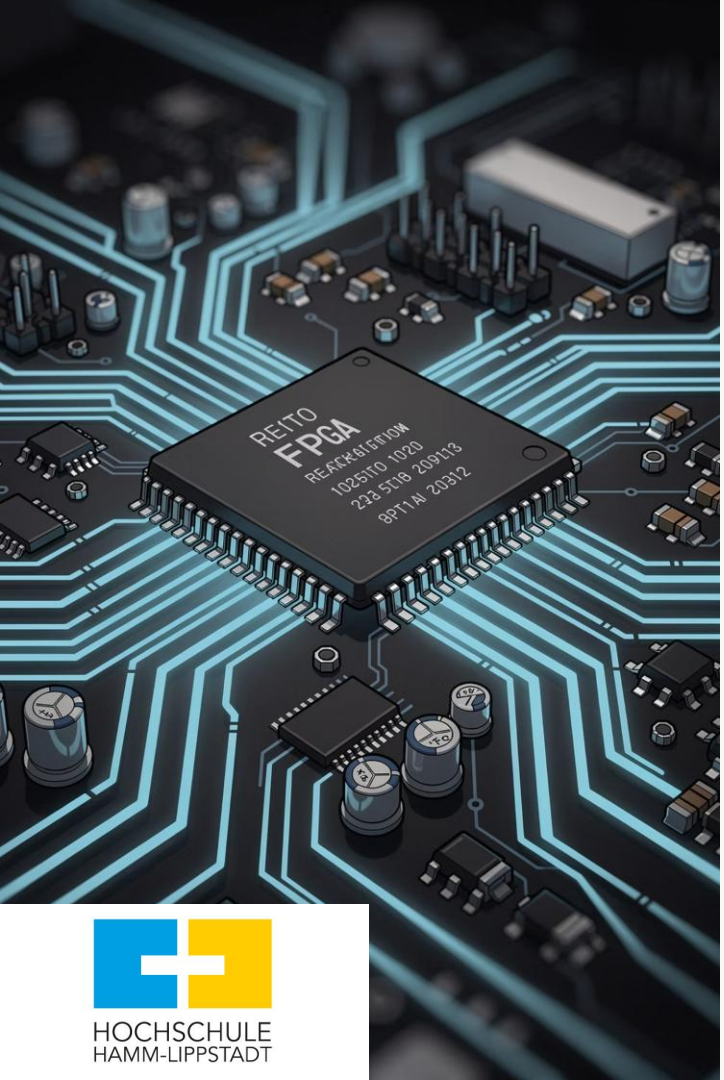


HOCHSCHULE
HAMM-LIPPSTADT

High-Level Synthesis with Game-Theoretic Scheduling

Power Optimization during Behavioral Synthesis

Presenter: Sumon Boiddo | **Institute:** Hochschule Hamm-Lippstadt

Scheduling

When operations run

Binding

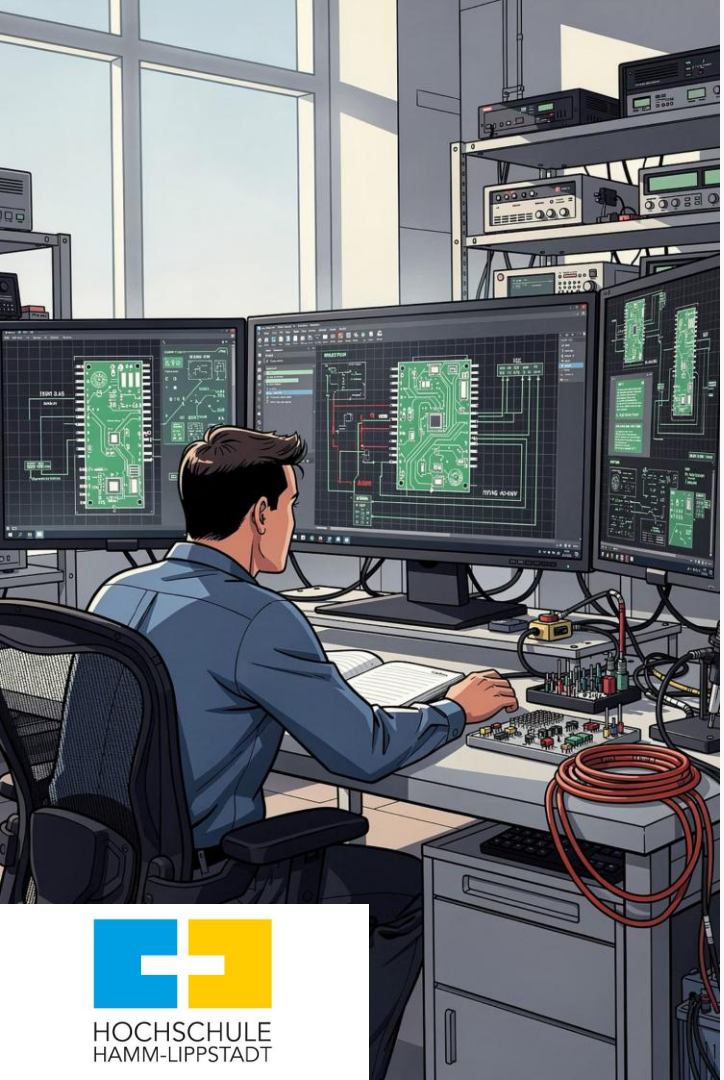
Resource-operation assignment

Nash Equilibrium

Stable, optimal decisions

Results

Measured power savings



HOCHSCHULE
HAMM-LIPPSTADT

What is High-Level Synthesis?

High-Level Synthesis (HLS) automatically converts algorithmic descriptions written in high-level languages such as C or C++ into register-transfer level (RTL) hardware designs. It eliminates the need for manual gate-level design, dramatically reducing development time and designer effort.

1

Scheduling

Assign operations to control steps (clock cycles)

2

Allocation

Determine number and type of hardware resources needed

3

Binding

Map operations to specific functional units and registers

Together, these three steps translate an abstract algorithm into a concrete, synthesizable hardware netlist ready for FPGA or ASIC implementation.

Why Power Optimization Matters

The Challenge

As silicon technology scales down, power consumption has become a first-class design constraint — not an afterthought. Every wasted switching event translates directly into heat, reduced battery life, and reliability degradation.

Key Consequences of High Power

- Faster battery drain in portable and IoT devices
- Increased heat generation requiring expensive cooling
- Higher packaging and thermal management costs
- Reduced circuit reliability and lifespan
- Thermal throttling, limiting peak performance

Embedded systems, sensors, wearables, and mobile SoCs are all critically affected by power budget decisions made during HLS.

The Core Problem in HLS

Naïve or suboptimal scheduling and binding decisions can significantly inflate power consumption. The root cause lies in unnecessary **switching activity** — every time a signal changes value, dynamic power is dissipated.

⏱️ Scheduling

Decides *when* each operation executes. Poor scheduling forces data to travel through extra logic, increasing transitions on interconnect and functional unit inputs.

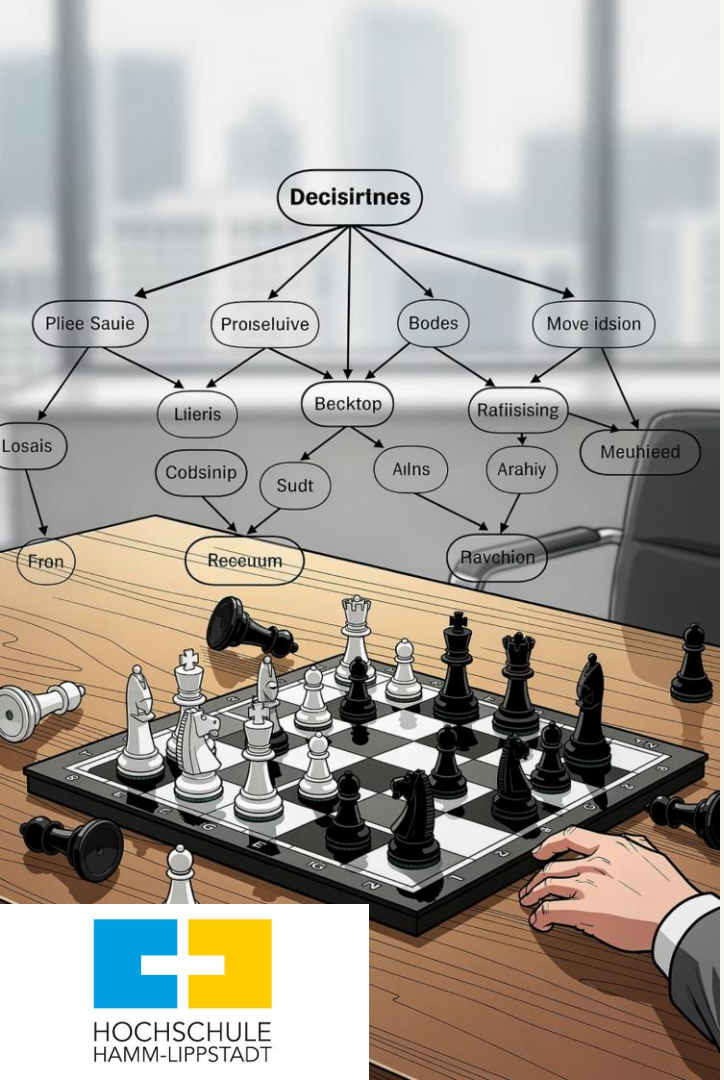
🔧 Allocation

Determines *how many* hardware resources are instantiated. Over-allocation creates idle units that still leak power; under-allocation causes congestion and pipeline stalls.

🔗 Binding

Maps operations to *specific* functional units and registers. Poor binding pairs operations with dissimilar inputs, maximizing bit-level transitions and power cost.

⚠ More switching activity → More dynamic power consumption. Optimal binding is the single largest lever for power reduction at the HLS stage.



HOCHSCHULE
HAMM-LIPPSTADT

Why Use Game Theory?

Game theory provides a mathematically rigorous framework for decision-making when multiple agents compete for resources under conflicting objectives — exactly the situation in HLS binding.

Players

Functional units (adders, multipliers, ALUs) act as rational players, each seeking to minimize their individual power cost when assigned an operation.

Strategies

Each player's strategy is a bid representing the estimated switching power cost of executing a given operation on that specific unit.

Equilibrium

Nash equilibrium identifies the globally stable assignment — no unit can unilaterally reduce system power by switching to a different operation.

Auction-Based Scheduling Mechanism

The game-theoretic scheduler operates like a **sealed-bid auction**: operations are tasks to be assigned, and functional units submit power-cost bids. The unit with the **lowest bid wins** the operation assignment, minimizing switching activity at each scheduling step.

Bidding Example — Addition Operation

Functional Unit	Power Bid (mW)	Result
Adder 1	8	❌ Not selected
Adder 2	5	❌ Not selected
Adder 3	3	✅ **Winner**

Adder 3 wins because it shares the most common inputs with the operation, reducing bit transitions and power cost.

Why This Works

The bid value is not arbitrary — it is computed from the **switching power model**, accounting for input similarity between the operation and the unit's previous assignment. Units that have recently processed similar data have lower switching costs and naturally win relevant tasks.

- Encourages data locality in binding
- Minimizes unnecessary bit-level transitions
- Scales naturally to multiple resource types

HOCHSCHULE
HAMM-LIPPSTADT

Nash Equilibrium — The Stable Solution

Nash equilibrium is the cornerstone of the game-theoretic approach. It defines a binding assignment where **no single functional unit can reduce the overall system power by unilaterally changing its assigned operation**.

Definition

A set of strategies (one per player/unit) from which no player benefits by deviating alone. The system has reached a globally consistent, self-enforcing solution.

In HLS Context

Each functional unit is assigned exactly the operation that minimizes mutual switching cost. Reassigning any single unit would only increase total power.

Guarantee

The equilibrium provides a *provably stable* and power-efficient binding. It replaces exhaustive search with a convergent iterative bidding process.

Nash equilibrium does not require a central controller — stability emerges naturally from the decentralized bidding interactions of the functional units.

HOCHSCHULE
HAMM-LIPPSTADT

Methodology: Step-by-Step Flow

Read DFG

Identify Ready Ops

X = A + B
Y = C + D
Z = X * Y

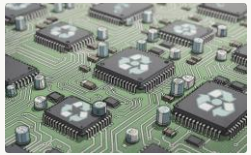
Equilibrium & Schedule

Units Submit Bids(Adder, Multiplier)

The algorithm processes the Data Flow Graph (DFG) **level by level**. Only operations whose predecessors have already been scheduled are considered "ready" and enter the bidding round. This ensures data dependencies are respected while minimizing switching activity at each step through the auction mechanism.

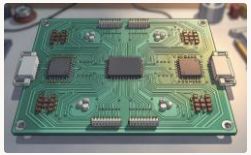
HOCHSCHULE HAMM-LIPPSTADT

Three Key Power Reduction Techniques



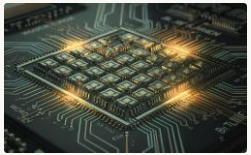
Functional Unit Sharing

The same hardware unit is reused across multiple operations with similar data patterns. Reuse reduces the number of active units and dramatically cuts switching activity by keeping input values stable between consecutive operations.



Path Balancing

Input signals to a functional unit are aligned to arrive simultaneously. Misaligned arrivals cause spurious transitions — the unit processes intermediate glitches before settling on the correct output, wasting power unnecessarily.



Register Assignment

Values stored in registers are carefully assigned to minimize the Hamming distance between successive values read from or written to the same register. Fewer bit flips per register access directly lowers dynamic power in the storage network.

Simple example:

Operation 1: A + B
Operation 2: A + C

X arrives in cycle 1
Y arrives in cycle 3

0000
0001

HOCHSCHULE
HAMM-LIPPSTADT

Mathematical Foundation

The binding cost model quantifies how much switching power a specific operation-to-unit assignment incurs, accounting for input commonality with neighboring operations:

Binding Cost Function

$$c_o(m, o) = P_{sw} - P_{inp}$$

P_sw — Switching power of the functional unit when executing operation o

P_inp — Power reduction gained from common or neighborhood inputs shared with adjacent operations

Global Optimization Objective

$$CO(A) = \min_{A} CO(A)$$

The algorithm searches across all feasible binding assignments **A** and selects **A*** — the assignment that achieves the minimum total binding cost across all scheduled operations.

In practice, the Nash equilibrium auction converges to A* efficiently without enumerating all possible assignments — unlike brute-force or ILP approaches.

HOCHSCHULE
HAMM-LIPPSTADT

Experimental Setup

Benchmark Circuits

01

Differential Equation

02

FIR Filter

03

IIR Filter

04

Lattice Filter

05

Elliptic Wave Filter

06

WAVE & Noise Cancel Filter

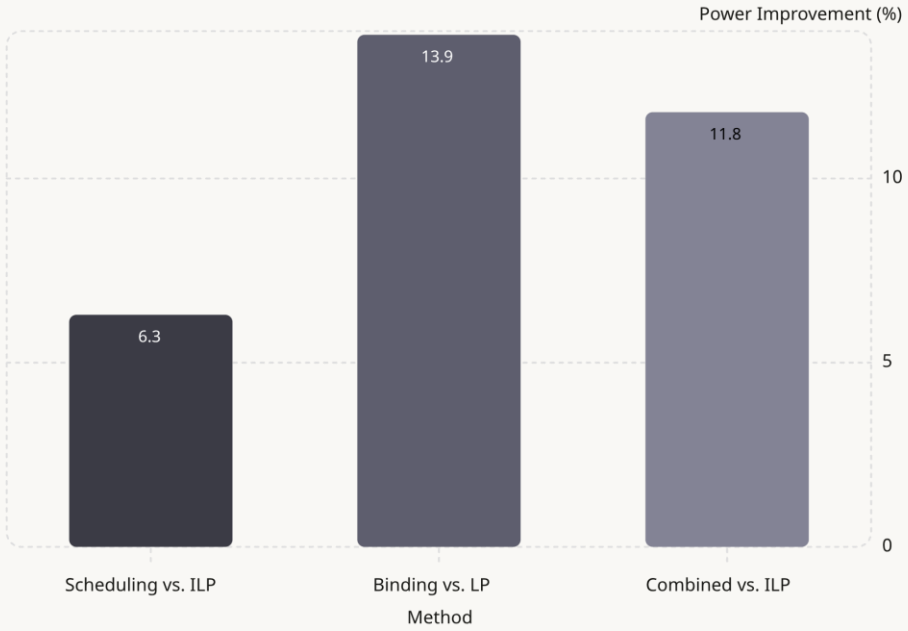
Simulation Parameters

- **100,000** input vectors per benchmark for statistical accuracy
- **16-bit two's complement** random integers as stimuli
- Power measured using post-synthesis gate-level simulation
- Compared against ILP-based scheduling and LP-based binding

Standard DSP benchmarks were chosen because they represent real-world compute-intensive applications where power efficiency is critical, such as signal processing in IoT and mobile devices.

HOCHSCHULE
HAMM-LIPPSTADT

Experimental Results



Key Findings

6.3% — Scheduling

Game-theoretic scheduling outperforms classical ILP-based scheduling on power.

13.9% — Binding

Auction-based binding yields the largest individual improvement vs. LP binding.

11.8% — Combined

Joint scheduling + binding delivers strong end-to-end power savings.

Chip area did not increase. Runtime remained comparable to traditional methods.

HOCHSCHULE
HAMM-LIPPSTADT

Conclusion & References

Summary

- HLS automates the translation of high-level code into optimized hardware
- Scheduling and binding are the dominant levers for power control
- Game theory models functional units as rational, power-minimizing players
- Nash equilibrium produces a stable, provably efficient assignment
- Results show **up to 13.9%** power reduction with no area penalty
- Limitation: computational complexity grows with circuit size; slight latency increase observed in some benchmarks

This approach is especially valuable for low-power embedded systems, IoT sensors, wearables, and mobile SoCs where every milliwatt matters.

References

[1] Murugavel & Ranganathan, 2003

A. K. Murugavel and N. Ranganathan, "A Game Theoretic Approach for Power Optimization During Behavioral Synthesis," *IEEE Trans. VLSI Systems*, vol. 11, no. 6, pp. 1031–1043, Dec. 2003.

[2] Sumon Boiddo, HSHL

S. Boiddo, "High-Level Synthesis with Game Theoretic Scheduling," Seminar Paper, Hochschule Hamm-Lippstadt

HOCHSCHULE
HAMM-LIPPSTADT

Thank You for Your Attention

HOCHSCHULE
HAMM-LIPPSTADT